

NORMALIZACION

Materia: Bases de Datos

7º Año Info

Contenido

Normalización.....	2
1. Eliminación de la redundancia de datos.....	2
2. Prevención de anomalías de actualización.....	2
3. Garantía de integridad y calidad	3
4. Mejora en la eficiencia y mantenimiento	3
Primera forma normal (1FN)	3
Ejemplos de aplicación de la 1FN.....	3
Ejercicios	5
Segunda Forma Normal (2FN)	9
Ejemplo de infracción de la 2FN	9
La solución: Descomposición.....	9
Caso especial: Claves Simples	10
Ejercicios	10
Tercera Forma Normal (3FN)	14
Ejemplo de infracción de la 3FN	14
La solución: Descomposición.....	14
Ejercicios	15

Normalización

La **normalización** es un proceso esencial en el diseño de bases de datos relacionales que consiste en aplicar una serie de reglas (formas normales) para asegurar que la estructura de las tablas sea la más adecuada. Su necesidad se fundamenta principalmente en los siguientes pilares:

1. Eliminación de la redundancia de datos

El objetivo primordial de la normalización es **evitar la duplicidad innecesaria de información**. Cuando una base de datos no está normalizada, un mismo dato puede aparecer repetido en múltiples lugares, lo que conlleva varios problemas:

- **Desperdicio de espacio:** Se utiliza más almacenamiento del necesario al guardar lo mismo varias veces.
- **Inconsistencia:** Si un dato cambia (por ejemplo, la dirección de un cliente) y está repetido en diez tablas, es muy probable que se actualice en algunas y se olvide en otras, rompiendo la "integridad" de la información.

2. Prevención de anomalías de actualización

Un diseño mal estructurado provoca que las operaciones básicas sobre los datos (insertar, borrar o modificar) sean problemáticas o incluso imposibles:

- **Anomalía de inserción:** Ocurre cuando no puedes añadir un dato nuevo porque te falta otro dato que no debería ser obligatorio (ej. no poder registrar un nuevo curso hasta que haya al menos un alumno inscrito).
- **Anomalía de borrado:** Sucede cuando, al eliminar un registro, pierdes involuntariamente información que querías conservar (ej. borrar al último alumno de un curso y que, por error, también desaparezca la descripción del curso de la base de datos).
- **Anomalía de modificación:** Se presenta cuando un cambio simple requiere actualizar múltiples filas, aumentando el riesgo de errores y el tiempo de proceso.

ID_EMP	NOMBRE	PROYECTO	DEPTO	LOCALIDAD
101	Juan	Proyecto Alfa	RRHH	Madrid
102	Ana	Proyecto Beta	RRHH	Madrid
103	Carlos	Proyecto Gama	Ventas	Barcelona
104	Juan	Proyecto Delta	RRHH	Madrid
105	Ana	Proyecto Épsilon	RRHH	Madrid

Redundancia: Desperdicio masivo de espacio de almacenamiento.

Anomalía de Actualización: Si RRHH se muda, hay que modificar 5 filas. Riesgo de inconsistencia.

Anomalía de Borrado: Si despedimos a este empleado, perdemos toda la información del proyecto.

3. Garantía de integridad y calidad

La normalización actúa como un "**sello de calidad**" para el diseño lógico. Al organizar los datos según sus dependencias funcionales, se asegura que cada tabla represente un solo concepto del mundo real (separación de conceptos). Esto produce:

- **Datos exactos:** Si el dato está bien una vez, está bien en todas partes.
- **Estabilidad:** Los esquemas normalizados son más flexibles y fáciles de extender ante futuros cambios en los requisitos del negocio.

4. Mejora en la eficiencia y mantenimiento

Aunque a veces se piensa que dividir tablas penaliza el rendimiento (porque requiere más "uniones" o JOINS), un diseño normalizado facilita el mantenimiento. Es mucho más sencillo **entender, documentar y modificar** una base de datos donde cada dato tiene un lugar único y lógico que una donde la información está mezclada de forma desordenada.

En resumen, la normalización es necesaria para transformar datos complejos en estructuras simples, consistentes y robustas, asegurando que la base de datos sea una **representación fiel y confiable** de la realidad de la empresa.

Primera forma normal (1FN)

La **Primera Forma Normal (1FN)** es el nivel básico de normalización y un requisito fundamental para cualquier base de datos relacional. Una relación se encuentra en 1FN si y solo si cada uno de sus atributos es **atómico**, lo que significa que los valores en las celdas de la tabla deben ser **indivisibles y simples**.

En términos prácticos, cumplir con la 1FN implica que:

- **No existen grupos repetitivos:** Un atributo no puede contener múltiples valores del mismo tipo en una sola fila.
- **Cada intersección de fila y columna contiene un solo valor:** No se permiten listas, conjuntos ni tablas anidadas dentro de un campo.
- **Debe existir una clave primaria:** La tabla debe tener una identificación única para cada registro y no puede haber filas duplicadas.

Ejemplos de aplicación de la 1FN

1. Eliminación de múltiples valores en una celda (Atributos multivaluados) Supongamos una tabla de **Alumnos** con un campo llamado "Teléfonos". Si en una sola celda se registran varios números (ej. "455-6985, 455-6853"), la tabla **no está en 1FN** porque el campo no es atómico.

DNI	Nombre	Dirección	Mail	Telefono
75895999	Perez. Lucas	Alvarez 55	perez@gmail.com	455-3566
75896998	Lopez. Maria	Roca 43	mar@gmail.com	455-6985

				455-6853
--	--	--	--	----------

- **Solución:** Se debe crear una tabla independiente de "Teléfonos" donde cada número sea un registro separado vinculado al alumno por su DNI, o bien generar filas adicionales en la tabla original para cada número, aunque esto último genera redundancia.

Tabla "Alumnos"

DNI	Nombre	Direccion	Mail
75895999	Perez. Lucas	Alvarez 55	perez@gmail.com
75896998	Lopez. Maria	Roca 43	mar@gmail.com

Tabla "Teléfonos"

DNI	Telefonos
75895999	455-3566
75896998	455-6985
75896998	455-6853

2. Descomposición de atributos compuestos Un campo "Nombre" que incluya el nombre de pila y los apellidos (ej. "Gervasio Perez Flores") puede violar la 1FN si el diseño requiere que estos datos sean tratados como unidades mínimas.

DNI	Nombre	Direccion	Mail
75895999	Perez. Lucas	Alvarez 55	perez@gmail.com
75896998	Lopez. Maria	Roca 43	mar@gmail.com

- **Solución:** El nombre debe dividirse en campos atómicos más pequeños como "Nombre", "Apellido Paterno" y "Apellido Materno" para que la información sea indivisible y fácil de manipular.

DNI	Apellido	Nombre	Direccion	Mail
-----	----------	--------	-----------	------

3. Eliminación de grupos repetitivos y tablas anidadas Si tenemos una tabla de **Compositores** con una columna "Obras" que contiene una lista de todas las sinfonías compuestas por esa persona, la estructura no es atómica.

Compositor	Obras
Beethoven	Sinf. 1, Sinf. 2, Sinf. 3
Mozart	Sinf. 40, Sinf. 41

- **Solución:** Se debe "aplanar" la relación, creando una fila por cada obra del compositor, asegurando que cada celda de la columna "Obras" contenga un solo título.

Compositor	Obras
Beethoven	Sinf. 1
Beethoven	Sinf. 2

Beethoven	Sinf. 3
Mozart	Sinf. 40
Mozart	Sinf. 41

Ya sé lo que estás pensando, ¿y ahora qué pasa con la clave principal?

● ¿Qué ocurre?

- **Compositor se repite** → ya no puede ser clave primaria
- Porque una clave primaria **no puede tener valores duplicados**

Solución 1: Clave primaria compuesta

Usas **más de una columna como clave primaria**:

(Compositor, Obra)

Esto significa:

- La combinación de ambos campos es única
- No puede haber dos filas con: Beethoven + Sinf. 1

Solución 2: Crear un ID único (esta es más común en el mundo real)

ID	Compositor	Obras
1	Beethoven	Sinf. 1
2	Beethoven	Sinf. 2
3	Beethoven	Sinf. 3
4	Mozart	Sinf. 40
5	Mozart	Sinf. 41

El ID va a ser un valor único que podés utilizar como PK.

Ejercicios

Ejercicio 1 – Teléfonos

Tabla original:

ID_Alumno	Nombre	Teléfonos
1	Juan Pérez	111-111, 222-222
2	Ana López	333-333

Consigna:

- Llevar la tabla a 1FN
- Definir la clave primaria

Ejercicio 2 – Cursos



Estudiante	Cursos
Pedro	Matemática, Física
Laura	Historia

Consigna:

- Transformar a 1FN
- Indicar si usarías clave compuesta o ID

Ejercicio 3 – Pedidos

ID_Pedido	Cliente	Productos
101	Carlos	Teclado, Mouse
102	Marta	Monitor

Consigna:

- Convertir a 1FN
- ¿Cuál sería la clave primaria?

Ejercicio 4 – Profesores

Profesor	Materias	Aulas
Gómez	BD, Programación	A1, A2
Pérez	Matemática	B1

Consigna:

- Normalizar a 1FN
- Explicar qué hiciste con las columnas múltiples

Ejercicio 5 – Facturación simple

NroFactura	Cliente	Productos	Cantidades
1	Juan	Pan, Leche	2, 1
2	Ana	Queso	1

Consigna:

- Llevar a 1FN

- Cuidado: hay **2 columnas con listas relacionadas**
- ¿Qué clave elegirías?

Ejercicio 6 – Películas

Película	Actores
Matrix	Reeves, Moss
Titanic	DiCaprio, Winslet

Consigna:

- Pasar a 1FN
- ¿Se repiten datos? ¿Importa en 1FN?

Ejercicio 7 – Sistema universitario

ID_Alumno	Nombre	Materias(nota)
1	Ana	Matemática(8), Física(7)
2	Luis	Historia(9)

Consigna:

- Identificar el problema
- Llevar a 1FN
- ¿Qué columnas nuevas aparecen?

Ejercicio 8 – Agenda de contactos

Nombre	Emails	Teléfonos
Juan	juan@gmail.com, juan@work.com	123, 456
Ana	ana@gmail.com	789

Consigna:

- Transformar a 1FN
- ¿Cuántas filas tendrás para Juan?

Ejercicio 9 – Verdadero/Falso

Indicar si cumple 1FN:

1. Una columna tiene valores separados por coma



2. Cada fila es única
3. Una celda contiene una lista
4. Cada valor es indivisible

Ejercicio 10 – Explicación

“¿Por qué una tabla con listas viola la 1FN?”

Segunda Forma Normal (2FN)

La **Segunda Forma Normal (2FN)** es el siguiente paso en el proceso de diseño y tiene como objetivo principal eliminar la redundancia que ocurre cuando una tabla tiene una **clave primaria compuesta** (formada por dos o más columnas).

Para que una relación esté en 2FN, debe cumplir con dos requisitos:

1. Debe estar previamente en **Primera Forma Normal (1FN)**.
2. Todos los atributos que no forman parte de la clave primaria deben tener una **dependencia funcional completa** de dicha clave.

En términos sencillos: si la clave es una combinación de campos (A + B), cualquier otro dato de la fila debe depender de **A y B juntos**, no solo de uno de ellos.

Ejemplo de infracción de la 2FN

Imagina una tabla que registra el inventario de una cadena de librerías:

Cód. libro (PK)	Cód. tienda (PK)	Cantidad	Dirección Tienda
342	12	3	Mayor 5
268	9	1	Paseo 10
181	9	7	Paseo 10

- **Clave Primaria:** (Cód. libro + Cód. tienda). Necesitamos ambos para saber cuántas unidades de un libro específico hay en una tienda específica.
- **Problema:** El campo "Dirección Tienda" depende únicamente del "Cód. tienda", no del libro. Esto genera redundancia (la dirección "Paseo 10" se repite para cada libro que haya allí) y **anomalías de actualización:** si la tienda 9 se muda, habría que cambiar la dirección en muchas filas, arriesgándonos a dejar datos inconsistentes.

La solución: Descomposición

Para normalizar esta tabla a 2FN, debemos **dividirla en dos**, separando los hechos que dependen de solo una parte de la clave:

1. **Tabla de Tiendas:** Donde la clave es solo "Cód. tienda".
 - (Cód. tienda, Dirección).
2. **Tabla de Stock:** Donde se mantiene la relación entre libros y tiendas.
 - (Cód. libro, Cód. tienda, Cantidad).

Caso especial: Claves Simples

Si una tabla ya está en 1FN y su **clave primaria es simple** (un solo campo, como un ID de usuario o DNI), entonces **automáticamente está en 2FN**. Esto es porque no es posible que un atributo dependa de "parte" de la clave si la clave no tiene partes.

Ejercicios

Ejercicio 1 – Cursos de alumnos

ID_Alumno	ID_Curso	Nombre_Alumno	Nombre_Curso
1	A	Juan	Matemática
1	B	Juan	Física
2	A	Ana	Matemática

Consigna:

- Identificar la clave primaria
- Detectar dependencias parciales
- Llevar la tabla a 2FN

Pista:

- ¿De qué depende Nombre_Alumno?
- ¿De qué depende Nombre_Curso?

Ejercicio 2 – Pedidos y productos

ID_Pedido	ID_Producto	Cliente	Precio
1	P1	Juan	10
1	P2	Juan	15
2	P1	Ana	10

Consigna:

- Determinar la clave (es compuesta)
- Identificar qué columnas dependen solo de una parte
- Normalizar a 2FN

Este es un caso típico donde:

- Cliente depende de pedido
- Precio depende del producto

Ejercicio 3 – Inscripciones

ID_Alumno	ID_Materia	Nombre_Alumno	Nombre_Materia	Nota
1	M1	Juan	BD	8
1	M2	Juan	Prog	7
2	M1	Ana	BD	9

Consigna:

- Identificar la clave primaria
- Detectar dependencias parciales
- Separar en tablas correctas

Ejercicio 4 – Biblioteca

ISBN	ID_Autor	Título	Nombre_Autor	Editorial
------	----------	--------	--------------	-----------

Supongamos que:

- Un libro puede tener varios autores
- PK = (ISBN, ID_Autor)

Consigna:

- ¿Qué atributos dependen de solo ISBN?
- ¿Qué atributos dependen de solo ID_Autor?
- Llevar a 2FN

Ejercicio 5 – Ventas

ID_Venta	ID_Producto	Nombre_Cliente	Nombre_Producto	Cantidad
----------	-------------	----------------	-----------------	----------

Consigna:

- Determinar la clave primaria
- Identificar dependencias parciales
- Normalizar a 2FN (crear tablas nuevas)

Ejercicio 6 – Películas



Título	Formato	Director	Duración
--------	---------	----------	----------

Supón:

- PK = (Título, Formato)

Consigna:

- ¿Director depende de toda la clave o solo de Título?
- ¿Duración depende de toda la clave?
- Normalizar

Ejercicio 7 – Proyectos

ID_Empleado	ID_Proyecto	Nombre_Empleado	Presupuesto_Proyecto	Horas
-------------	-------------	-----------------	----------------------	-------

Consigna:

- Identificar la clave primaria
- Detectar dependencias parciales
- Diseñar tablas en 2FN

Ejercicio 8 – Hoteles

ID_Reserva	ID_Habitación	Cliente	Tipo_Habitación	Noches
------------	---------------	---------	-----------------	--------

Consigna:

- Detectar dependencias parciales
- ¿Qué depende de reserva?
- ¿Qué depende de habitación?
- Llevar a 2FN

Ejercicio 9 – Verdadero/Falso

Indicar si viola 2FN:

1. Una tabla tiene clave simple
2. Una columna depende de toda la clave
3. Una columna depende solo de una parte
4. Todos los atributos dependen de la clave completa

Ejercicio 10 – Explicación

“¿Por qué una dependencia parcial causa redundancia?”

Tips

Siempre sigan este proceso:

1. ✓ Verificar que esté en 1FN
2. ✓ Identificar clave primaria (¿es compuesta?)
3. ✓ Pregunta clave:

“¿Este atributo depende de TODA la clave o solo de una parte?”

4. ✓ Separar tablas

Truco didáctico clave

Clave: (A, B)

Si algo depende solo de A → ✗ mal (2FN)

Si algo depende solo de B → ✗ mal (2FN)

Si depende de A+B → ✓ bien

Tercera Forma Normal (3FN)

La **Tercera Forma Normal (3FN)** tiene como objetivo principal eliminar las **dependencias transitivas**. Esto significa que ningún atributo que no sea parte de la clave primaria puede depender de otro atributo que tampoco sea clave.

Para que una relación esté en 3FN, debe cumplir con dos condiciones:

1. Debe estar previamente en **Segunda Forma Normal (2FN)**.
2. No deben existir dependencias transitivas entre sus atributos; es decir, todos los atributos no clave deben depender **directamente** (y solo) de la clave primaria.

Como dice la frase popular en el diseño de bases de datos: *"Cada columna debe depender de la clave (1FN), de toda la clave (2FN) y **nada más que de la clave (3FN)**".*

Ejemplo de infracción de la 3FN

Supongamos una tabla para gestionar el **Hábitat de Inquilinos**:

DNI Inquilino (PK)	Edificio	Alquiler Mensual
11111111A	Edificio Alba	500 €
22222222B	Edificio Alba	500 €
33333333C	Edificio Betania	750 €

- **Estado actual:** La tabla está en 2FN porque la clave es simple (DNI).
- **Problema:** Existe una **dependencia transitiva**. El "Alquiler Mensual" no depende directamente del inquilino, sino del "Edificio". Si sabemos el edificio, ya sabemos el alquiler, independientemente de quién viva allí.
- **Anomalía:** Si un edificio se queda vacío, podríamos perder la información de cuánto cuesta su alquiler si borramos al último inquilino (anomalía de borrado).

La solución: Descomposición

Para normalizar esta estructura a 3FN, debemos dividir la tabla original en dos, separando los conceptos:

1. **Tabla Alquileres:** Donde la clave es el edificio.
 - (Edificio, Alquiler).
2. **Tabla Habita:** Donde se vincula al inquilino con su edificio.
 - (DNI Inquilino, Edificio). El campo "Edificio" actúa ahora como clave ajena (FK).

De esta forma, si el precio del alquiler de un edificio cambia, solo lo actualizamos en un registro, evitando inconsistencias.

Ejercicios

Ejercicio 1 – Empleados

ID_Empleado	Nombre	ID_Departamento	Nombre_Departamento
1	Juan	D1	Ventas
2	Ana	D2	Sistemas

Consigna:

- Identificar la clave primaria
- Detectar dependencia transitiva
- Llevar la tabla a 3FN

💡 Pista:

- ID_Empleado → ID_Departamento
 - ID_Departamento → Nombre_Departamento
- Entonces hay dependencia transitiva

Ejercicio 2 – Cursos

ID_Curso	Nombre_Curso	ID_Profesor	Nombre_Profesor
----------	--------------	-------------	-----------------

Consigna:

- ¿Qué depende de qué?
- ¿Hay dependencia transitiva?
- Normalizar a 3FN

Ejercicio 3 – Estudiantes y ciudades

ID_Alumno	Nombre	Código_Postal	Ciudad
-----------	--------	---------------	--------

Supón:

- Código_Postal → Ciudad

Consigna:

- Identificar la clave primaria



- Detectar dependencia transitiva
- Llevar a 3FN

Ejemplo clásico:

ID → CódigoPostal → Ciudad

Eso es dependencia transitiva

Ejercicio 4 – Productos

ID_Producto	Nombre	ID_Categoria	Nombre_Categoria
-------------	--------	--------------	------------------

Consigna:

- Detectar dependencia transitiva
- Separar tablas



Esto produce redundancia si no se separa

Ejercicio 5 – Facturación

ID_Factura	ID_Cliente	Nombre_Cliente	Ciudad_Cliente
------------	------------	----------------	----------------

Supón:

- ID_Cliente → Nombre_Cliente, Ciudad_Cliente

Consigna:

- Identificar dependencias
- Encontrar la transitiva
- Diseñar modelo en 3FN

Ejercicio 6 – Películas

Título	Director	Nacionalidad_Director	Duración
--------	----------	-----------------------	----------

Supón:

- Director → Nacionalidad

Consigna:

- Detectar dependencia transitiva
- Llevar a 3FN

Este es el típico caso:

Título → Director → Nacionalidad

Ejercicio 7 – Inscripciones completas

ID_Alumno	ID_Materia	Nombre_Alumno	Nombre_Materia	Profesor	Ciudad_Profesor
-----------	------------	---------------	----------------	----------	-----------------

Supón:

- ID_Alumno $\rightarrow$ Nombre_Alumno
- ID_Materia $\rightarrow$ Nombre_Materia, Profesor
- Profesor $\rightarrow$ Ciudad_Profesor

Consigna:

- Identificar todas las dependencias
- Detectar parciales y transitivas
- Llevar hasta 3FN

Ejercicio 8 – Ventas

ID_Venta	ID_Producto	Nombre_Producto	ID_Proveedor	Nombre_Proveedor
----------	-------------	-----------------	--------------	------------------

Supón:

- ID_Producto $\rightarrow$ Nombre_Producto, ID_Proveedor
- ID_Proveedor $\rightarrow$ Nombre_Proveedor

Consigna:

- Detectar dependencia transitiva
- Diseñar tablas en 3FN

Ejercicio 9 – Verdadero/Falso

1. Una dependencia transitiva es cuando $A \rightarrow B$ y $B \rightarrow C$
2. En 3FN no puede haber dependencias transitivas
3. Si un atributo depende de otro no clave
4. 3FN elimina dependencias parciales

Ejercicio 10 – Explicación

Explica con tus palabras qué es una dependencia transitiva y por qué es un problema

Tips para tener en cuenta

Checklist:

1. ¿Está en 1FN? 

2. ¿Está en 2FN? 

3. ¿Algún atributo depende de otro atributo no clave?  → problema

 Regla de oro de 3FN:

 "Todo atributo NO CLAVE debe depender SOLO de la clave"